Week 7: Web Scraping and Automation Pipeline

Key Concepts:

- **Workflow Automation:** Moving beyond analyzing pre-existing CSV files to **creating your own dataset** from the web. This involves a multi-stage pipeline: Accessing the site $\rightarrow$ Crawling for links $\rightarrow$ Downloading files $\rightarrow$ Extracting specific data.
- **Headless Browsing:** Understanding that modern websites require a real browser (running invisibly/headlessly) to handle JavaScript, cookie consent banners, and dynamic content, which simple "download" commands cannot handle.
    - **Note:** JavaScript is the programming language that makes websites interactive (like loading new posts as you scroll); without a browser to actually "run" this code, the data often doesn't exist to be downloaded.
- **Anti-Bot Evasion:** The challenge of making an automated script appear "human" to avoid being blocked by security systems (e.g., Akamai). Strategies include managing User Agents, saving/loading Cookies, and mimicking human wait times.
    - **Note:** A User Agent is the "ID card" a browser shows to a website (e.g., "I am Google Chrome on Windows"); automated scripts must fake this ID so the website doesn't recognize them as a robot program.
- **Unstructured Data & OCR:** Dealing with data that isn't in rows and columns. This includes parsing HTML for links and handling PDFs that are "scanned images" (requiring **Optical Character Recognition** or OCR) versus those with embedded text layers.
- **Asynchronous Execution:** The concept that browsing the web involves "waiting" (for pages to load), allowing Python to perform other tasks or handle the browser efficiently using `async` and `await` keywords (managing "multitasking" without freezing the program).

Coding Techniques:

- **Session Persistence:** Saving the state of a browser (cookies and local storage) to a JSON file so subsequent scripts can "log in" automatically or bypass pop-ups.
    - **Note:** If you only load the cookies but forget the Local Storage, the website might "remember" who you are (via cookies) but "forget" that you already dismissed the annoying pop-up (via Local Storage).
- **Recursive Crawling:** The logic of visiting a page, collecting links, and then visiting *those* links up to a specific depth, while filtering for relevant topics (e.g., "Sustainability").
- **String Normalization:** Advanced text cleaning techniques to ensure keyword matching works regardless of formatting (e.g., treating "Water," "water," and "wáter" as the same word by stripping accents and punctuation).
- **Data Ledger:** Using **Pandas** not for analysis, but as a "checklist" (`df_DL.csv`) to track which

files have already been downloaded or processed, preventing duplicate work if the script is restarted.

Packages Introduced:

- `playwright`**:** A powerful library used to control a web browser programmatically. It handles clicking buttons (like "Accept Cookies"), taking screenshots for debugging, and navigating pages.
- `pymupdf` **(fitz):** Used for opening PDF files and extracting text or page images programmatically.
- `pytesseract`**:** An OCR tool used to "read" text from PDFs that are essentially just scanned images.
- `asyncio`**:** Python's built-in library for managing concurrent tasks, essential for running Playwright without freezing the notebook.
- `urllib.parse`**:** A utility for breaking down web addresses (URLs) into their components (domain, path, query) to clean and validate links.